Herramientas de ingeniería

Proyecto Grupal 1: sistema distribuido de reducción adaptativa de ruido

N. Florez, T. Cajiao, S. Robles, S. Boza
CE 1110 Análisis de Señales Mixtas, II Semestre 2026

1. Selección de técnicas, recursos y herramientas

Las herramientas se eligieron según las variables del problema: frecuencia de muestreo y aliasing, latencia por bloque, memoria del microcontrolador, integridad del enlace y posibilidad de verificar el firmware contra una referencia.

Categoría	Herramienta o recurso	Variable que atiende
Modelado y simulación	Python 3, NumPy, SciPy, Matplotlib [1, 2], Jupyter	Diseño FIR/IIR, DFT directa frente a FFT, métricas y figuras reproducibles con semillas fijas.
Firmware de adquisición	Pico SDK en C con ADC, DMA y TinyUSB [3, 4]	Muestreo continuo a 16 kHz sin pérdidas, quitar DC, Hann y FFT de diagnóstico.
Firmware de filtrado	ESP-IDF 5.5 (C, FreeRTOS) desde PowerShell, SPI esclavo, DAC continuo, ESP Timer [5, 6]	Dos rutas independientes en un microcontrolador de doble núcleo; tiempo por bloque medido en μs.
Comunicación	SPI 2 MHz con trama propia y CRC-8 [7]; USB serie a 1 Mbaud	Delimitación de bloques, detección de errores y pérdidas.
Datasheets	RP2040 (ADC), ESP32 (DAC 8 bits), MAX4466 [8], LM358 [9], nota de Sallen-Key [10]	Rango de entrada 0–3.3 V, resolución, alimentación y diseño del filtro de reconstrucción.
Instrumentos	Multímetro digital ANENG DM850	Alimentación de 5 V del LM358 y tensiones DC en sus pines durante la ejecución, de las que depende el filtro analógico.
Verificación	pytest, GCC, scripts de prueba en Python	Más de 150 pruebas en Python (modelo, núcleo C de filtros frente a SciPy, trama SPI, protocolo PC y cierre del DAC) y pruebas en C del receptor (DC, Hann, FFT frente a DFT, CRC-8).
Medición con la Pico	*pico_instrumento.py*, *capturas_pico.py*, *demo analogico*	ADC1 por USB: 12 bloques separados por punto, amplitudes de GPIO25 frente a pin 7 del LM358.
Validación independiente	*validar*, *concordancia.py*; esptool	Mismo PCM16 contra Python, criterio digital ≥ 90 %; diagnóstico del puente USB.
Documentación y gestión	Git (master/development, ramas por función), Mermaid, LaTeX IEEEtran	Trabajo incremental, diagramas versionados y artículo.
Inteligencia artificial	OpenAI Codex; Anthropic Claude	Desarrollo y depuración de bloques de código; investigación de fuentes y formato de documentación.

2. Aplicación de las herramientas al problema

El trabajo siguió una cadena en la que cada etapa se valida con la anterior:

1. **Modelo de referencia.** El cuaderno y `prototipo.py` generan los casos con semillas, implementan la DFT por definición y la comparan con la FFT, diseñan un FIR Kaiser y tres IIR, y calculan MSE, SNR, energía y tiempo. Los parámetros se leen de archivos JSON, de modo que cada figura y tabla del artículo se regenera con un comando.
2. **Paso de Python a C.** `exportar_coeficientes.py` conserva los 75 coeficientes FIR en float32. Las señales se transfieren desde CSV como PCM16 al procesador de registros; las pruebas usan la misma entrada cuantizada que la placa.
3. **Verificación cruzada.** Las pruebas compilan con GCC el mismo núcleo C que corre en la ESP32 y lo comparan con Python: error por muestra $\leq 3 \times 10^{-6}$ y diferencias de métricas ≤ 0.03 dB. En la placa, la SNR de salida coincidió con Python sobre la misma entrada PCM16: diferencia FIR 3.3×10^{-5} dB y espectral 8.2×10^{-8} dB; concordancias de 100 y 99.99966 %, respectivamente.
4. **Cadena física y adquisición de datos.** Los datasheets fijaron el rango del ADC (0–3.3 V, 12 bits) y del DAC (8 bits, GPIO25). Con el multímetro ANENG DM850 se verificó, con el sistema en ejecución, que el LM358 recibe 5 V, condición para que el filtro analógico funcione; además, la entrada no inversora (pin 3) y la salida

(pin 1) del primer amplificador marcaron 0.815 V. Para el filtro analógico, la Pico se reutilizó como instrumento: *pico_instrumento.py* coordina USB y *capturas_pico.py* interpreta ADC1. La demo mide los mismos tonos antes (GPIO25) y después (pin 7) sin concatenar huecos. El modelo usa 22 nF y retención DAC; los tres tonos útiles cumplen y 4 kHz queda inválido por variabilidad de 19.3 %. El comando *validar* reprocesa en la PC la misma entrada guardada con los filtros de Python y calcula la concordancia con la salida de la ESP32, sin reemplazar el procesamiento de la placa. Con esptool se comprobó que el puente USB no respondía a una tasa superior, por lo que el enlace quedó en 1 Mbaud. La compilación, la programación y la ejecución se hicieron desde PowerShell; la PC solo recibe y muestra salidas. Los scripts llevan los datos del circuito a la PC: `graficar_microfono.py` muestra en vivo la forma de onda y el espectro que captura la Pico; `sesiones.py` configura parámetros sin recompilar, ordena capturas y evaluaciones en la ESP32, guarda entrada, salida FIR y salida espectral como WAV con `sesion.json`, métricas y registro de eventos, y verifica integridad con CRC e índices. *respuesta_fir_medida.py* compara los WAV de placa con los coeficientes; *resumen_pruebas.py* produce medias, desviaciones y tabla LaTeX por condición a partir de las sesiones completas, como exige el informe de repeticiones. No califica automáticamente el sistema. Registros completos locales y tablas/figuras seleccionadas para la entrega se mantienen separados.

5. **Uso de IA.** Codex se usó para escribir y depurar bloques de código (controladores, protocolo y pruebas), y Claude para buscar fuentes, revisar la coherencia entre código y documentación y dar formato a los documentos. Toda salida se aceptó solo después de revisarla, compilarla y pasar las pruebas automáticas; los valores numéricos del artículo provienen de las ejecuciones, no de la IA.

3. Adaptación de las técnicas al problema

- **FFT propia.** Se implementó una FFT radix-2 iterativa en C estándar, sin dependencias del SDK, para poder probarla en la PC; se validó contra NumPy.
- **FIR por bloques con estado.** El FIR se adaptó a un anillo de 75 muestras que conserva el historial entre bloques de 256, y la comparación compensa su retardo fijo de 37 muestras en lugar de buscar el desfase que mejor puntúe.
- **Espectral con memoria acotada.** La superposición y suma usa solo tres buffers de 1024 muestras; el umbral relativo se recalcula por bloque y protege la banda útil. Sus parámetros cambian en ejecución, como exige la defensa.
- **Protocolo propio.** La trama SPI (cabecera, identificador, 256 float32, CRC-8) y la línea READY se diseñaron para que la ESP32 detecte errores, saltos y colas llenas y aborte con evidencia, en vez de procesar datos corruptos.
- **Herramientas de compilación.** Se creó `ejecutar_idf.ps1`, que compila desde una copia sin tildes porque ESP-IDF falla con rutas como “generación_señales”.
- **Filtro analógico.** El Sallen-Key se dimensionó con la nota de TI para un corte de 2283 Hz con 22 nF, el valor comercial disponible más cercano a los 20 nF de un Butterworth exacto.

Referencias

[1] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[2] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.

[3] Raspberry Pi Ltd., “Raspberry Pi Pico-series C/C++ SDK.” [En línea]. Disponible: `https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf`

[4] Raspberry Pi Ltd., “RP2040 Datasheet,” sec. 4.9. [En línea]. Disponible: `https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf`

[5] Espressif Systems, “ESP-IDF Programming Guide v5.5.” [En línea]. Disponible: `https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/`

[6] Espressif Systems, “ESP32 Series Datasheet,” 2025.

[7] W. W. Peterson y D. T. Brown, “Cyclic codes for error detection,” *Proc. IRE*, vol. 49, no. 1, pp. 228–235, 1961.

[8] Maxim Integrated, “MAX4465–MAX4469 Micropower, Low-Noise Microphone Preamplifiers,” datasheet, 2017.

[9] Texas Instruments, “LM358, LM258, LM2904: Industry-Standard Dual Operational Amplifiers,” SLOS068AB, oct. 2024.

[10] J. Karki, “Analysis of the Sallen-Key Architecture,” Texas Instruments, SLOA024B, 2002.